

TOWSON UNIVERSITY
COLLEGE OF GRADUATE STUDIES

AI-ASSISTED ADOPTION OF NIX FLAKES FOR REPRODUCIBLE
DEVELOPMENT ENVIRONMENTS IN COMPUTER SCIENCE
EDUCATION

by

Luis Dale Gascon

A thesis

Presented to the faculty

of Towson University

in partial fulfillment

of the requirements for the degree

Master's of Computer Science

Department of Computer and Information Sciences

Towson University,

Towson, Maryland

21252

December



Student's Name

Luis Dale Gascon

Chairperson, Thesis Committee

Signature

DocuSigned by:

Sajidul Islam

A1834C6028884BD...

Md Sajidul Islam Sajid

Typed name

Note: Please attach a description of the affiliation and credentials of any non- Towson University members of the Committee, and the members' *curriculum vitae*.

Member

Signature

Signed by:

Sidd Kaza

1C79FD345A714C7...

Sidd Kaza

Typed name

Member

Signature

DocuSigned by:

Blair Taylor

F736E4AFF5A443D...

Blair Taylor

Typed name

Approved by

Graduate Program Director

Signature

Signed by:

Ziyang Tang

C6D51D455F89448...

02/02/26

Date

Department Chairperson

Signature

DocuSigned by:

Michael McQuinn

4FA74552EDAD4F1...

02/02/26

Date

☒ Graduate Faculty Status verified for all committee members

Dean of Graduate Studies

Signature

Signed by:

David R Ownby

0109A385FE4043B...

02/03/26

Date

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to Dr. Taylor and Dr. Kaza for their continuous support in opportunities and encouragement to go beyond my comfort zone.

I am also deeply grateful for my parents for their unconditional belief in my potential and their steadfast support throughout my college journey.

ABSTRACT

AI-ASSISTED ADOPTION OF NIX FLAKES FOR REPRODUCIBLE DEVELOPMENT ENVIRONMENTS IN COMPUTER SCIENCE EDUCATION

Luis Dale Gascon

Inconsistent development environments pose substantial friction for software engineers and development teams, frequently consuming days of onboarding time and shifting engineering focus from feature delivery to environment debugging. In computer science education, this challenge is acutely amplified. When students must imperatively configure their personal machines for diverse coursework, valuable lecture and lab hours are often sacrificed to troubleshooting dependency conflicts. The myriad combination of student hardware, operating systems, and local configuration states turns instructors and teaching assistants into ad-hoc technical support providers, creating severe barriers to student engagement and learning.

This thesis proposes an automated framework that leverages agentic large language models (LLMs) to synthesize reproducible development environments using Nix Flakes from instructors' natural language requirements or to infer requirements from a project's source code. Because environment synthesis is a recurrent task across semesters, we designed and implemented a specialized custom agent skill that provides an agent with domain-specific flake templates, structured metadata, and automated validation heuristics to repeatedly perform the task with the proper context. To prevent package hallucination within the massive Nixpkgs ecosystem, the

agent reaches out to a Model Context Protocol (MCP) server to retrieve real-time package metadata and configuration schemas directly from NixOS search indexes. Synthesized flakes are verified through a deterministic evaluation pipeline using native Nix tooling, enabling the agent to autonomously interpret compiler and evaluation error traces to self-correct invalid specifications. Furthermore, this study looks into the computational overhead between docker containers and Nix, demonstrating that declarative Nix Flake environments offer substantial memory savings, disk deduplication, and reduced startup latency compared to DevContainers on resource-constrained student laptops.

TABLE OF CONTENTS

List of Tables	ix
List of Figures	x
Chapter I. Introduction.....	1
A. The Reproducibility Issue in Computer Science Education.....	1
B. Virtualization versus Declarative Configuration: The Architectural Dilemma.....	2
C. AI-Assisted Synthesis: Overcoming the Usability Barrier	4
D. Research Objectives and Contributions	5
E. Organization of the Thesis	7
Chapter II. Literature Review.....	9
A. Standardizing Educational Environments via Virtualization and Containers.....	9
B. Synthesis of Infrastructure as Code Using Large Language Models...	10
C. Reproducible Challenge and Lab Environments for Cybersecurity Education	11
D. Declarative and Functional Package Management in Education	12
E. Agentic Skills and Tool-Augmented Assistance.....	13
F. Summary of Research Gaps.....	14
Chapter III. Research Questions and Proposed Architecture	15
A. Research Questions	15

B. Proposed Solution: The AI-Assisted Flake Synthesis Architecture ...	16
Chapter IV. Experimental Methodology and Test Setup	18
A. Agentic Flake Synthesis Test Design	18
1) Real-Time Package Retrieval via MCP-NixOS	18
2) Agent Harness and Model Configurations	19
3) Input Dataset Curation	19
4) Input Data Format	20
5) Specialized Agent Skill Design	20
6) Evaluation Metrics and Pass@K Formulation	22
B. Clean-Host Reproducibility Methodology	23
Chapter V. Empirical Results and Analysis	25
A. Flake Synthesis Accuracy and Model Comparison	25
B. Tool Invocation and Telemetry Analysis	25
C. Clean-Host Reproducibility Results - TODO	27
Chapter VI. Security Considerations and Practical Adoption	28
A. Security Considerations	28
B. Practical Adoption of Nix	28
Chapter VII Conclusion	30
A. Summary of Contributions	30
B. Synthesis of Research Questions	31
C. Limitations and Future Work	32
Appendix A Appendix Title Here	34

Appendix BAppendix Title Here.....	35
References	36

LIST OF TABLES

LIST OF FIGURES

Fig. 1	High-Level End-to-End Architecture of the Autonomous <code>flake.nix</code> Synthesis Pipeline	17
Fig. 2	Phase 1: Project Ingestion and Context Discovery Subsystem Across Greenfield and Brownfield Projects.....	17

CHAPTER I.

INTRODUCTION

A. The Reproducibility Issue in Computer Science Education

Software Configuration Management (SCM) has long served as a critical pillar of modern software engineering, ensuring consistency and deterministic reproducibility across development, testing, and deployment cycles [1]. In commercial software engineering, misconfigured developer machines frequently trigger the classic “works on my machine” syndrome, where subtle variances in operating system libraries, compiler revisions, or environment variables introduce non-deterministic bugs. Enterprise organizations systematically mitigate this friction through dedicated platform teams, standardized developer virtual machines, and automated continuous integration pipelines. In contrast, academic computer science programs operate under severe resource constraints and lack the institutional infrastructure required to maintain homogeneous client computing environments [2].

In modern computer science courses, students arrive in classrooms with a highly heterogeneous array of personal devices spanning macOS, disparate Linux distributions, and Microsoft Windows [3]. Introductory and advanced courses alike demand specialized toolchains, language runtimes, software development kits (SDKs), and external libraries. Historically, instructors have relied on imperative, manual installation guides distributed during the opening weeks of the semester. This approach invariably consumes substantial lecture and laboratory hours as

teaching assistants and faculty diagnose idiosyncratic system failures, operating as impromptu technical support staff rather than focusing on foundational computer science concepts. Consequently, configuration friction becomes an artificial barrier to student engagement and academic equity throughout the semester.

The degree of “futz” or the tinkering of a system’s state depends on a user’s general expertise in their OS and the installation process. [4]

B. Virtualization versus Declarative Configuration: The Architectural Dilemma

The root cause of configuration fragility lies in the imperative architecture of conventional operating system package managers. Tools such as Debian’s APT, Arch Linux’s Pacman, and macOS Homebrew mutate a shared global filesystem hierarchy, modifying directories such as `/usr/bin` and `/usr/lib` in place [5]. Because packages share mutable namespaces, upgrading a library required for one course frequently breaks the dependencies of another course, producing a web of dependency conflicts. Furthermore, because imperative package managers do not preserve transactional execution histories, rolling back failed upgrades to a known working state is not feasible.

To overcome the fragility of mutable host filesystems, the software industry widely adopted containerization via Docker and Open Container Initiative (OCI) Development Containers (DevContainers) [3]. DevContainers isolate the application runtime, compiler toolchains, and editor extensions within a sandboxed Linux container, ensuring that students execute assignments in an identical environment regardless of host operating system. However, this isolation comes at a steep

computational cost on consumer hardware. On non-Linux platforms such as macOS and Windows, containers require a background Linux virtual machine hypervisor, which persistently consumes between 1.8 GB and 2.4 GB of idle random-access memory (RAM) [6]. In the wake of global memory supply shortages and surging semiconductor prices, such heavy memory footprints strain student laptops, causing sluggish performance and thermal throttling when running concurrently with modern development environments and web browsers [7]. Additionally, container images duplicate common base layers across multiple courses, resulting in severe disk bloat.

In contrast to process virtualization, purely functional package management offers a lightweight, host-native alternative [8]. The Nix package manager treats software builds as mathematical pure functions, storing every built package under a unique cryptographic hash in an immutable directory known as the Nix store (`/nix/store`) [9]. Nix Flakes standardize this model by providing a hermetic interface governed by an explicit `flake.nix` specification and a pinned `flake.lock` dependency file [10]. Through ephemeral development shells (`devShells`), students can activate a fully configured, isolated development environment with a single shell invocation (`nix develop`), incurring zero hypervisor memory overhead while executing natively on the host kernel [11].

Despite these compelling architectural advantages, Nix Flakes have faced an acute adoption chasm in educational institutions [12]. The Nix expression language is a domain-specific, lazy functional programming language that departs radically from conventional imperative scripting. For students and instructors unfamiliar with

functional paradigms, deciphering Nix syntax, navigating complex attribute sets, and interpreting cryptic evaluation stack traces presents an overwhelming cognitive barrier. As a result, educational institutions often remain tethered to resource-heavy containerized workflows despite their recognized performance deficiencies.

C. AI-Assisted Synthesis: Overcoming the Usability Barrier

Recent breakthroughs in Large Language Models (LLMs) and agent harness architectures present an unprecedented opportunity to eliminate the cognitive barrier associated with declarative systems [13]. Rather than requiring instructors and students to compose complex Nix expressions manually, an autonomous agent can ingest natural language assignment requirements or analyze unconfigured source code repositories to synthesize valid `flake.nix` configurations automatically. By abstracting the syntax and semantics of the underlying configuration language, agentic workflows can democratize access to purely functional development environments.

However, deploying off-the-shelf, general-purpose LLMs for Nix synthesis exposes critical limitations. Because the Nix expression language represents a minute fraction of public code repositories compared to ubiquitous languages such as Python or C++, general-purpose models exhibit severe package hallucination rates [14]. When prompted to generate Nix environments, models frequently hallucinate non-existent package attribute names, reference deprecated configuration schemas, or default to imperative package installations on the host system.

To ground the agent in domain-specific best practices, our framework introduces a specialized custom agent skill named `course-flake-generator` [15, 16]. The skill encapsulates modular `devShell` templates, heuristic rule sets for detecting project requirements, and automated validation routines. By loading this structured context dynamically, the agent transitions from a generic language predictor into a specialized Nix configuration engineer capable of maintaining consistent behavioral patterns across diverse assignment prompts.

To eliminate package hallucination entirely, the agent is coupled with a dedicated Model Context Protocol (MCP) server, `mcp-nixos` [17]. The MCP server connects the agent harness directly to the real-time Elasticsearch index powering `search.nixos.org`, enabling live verification of package attributes across the 139,000+ packages in the Nixpkgs registry. Furthermore, our pipeline exploits the deterministic nature of Nix evaluation: when a generated flake contains syntax or typing errors, the compiler diagnostics from `nix flake check` are captured and fed back into the agent harness in a closed-loop recovery cycle, allowing the agent to self-correct invalid declarations prior to student deployment.

D. Research Objectives and Contributions

This thesis addresses the fundamental tension between computational efficiency and usability in educational development environments. By automating the synthesis and validation of Nix Flakes through an agentic AI pipeline, we demonstrate that educational institutions can achieve lightweight, host-native

reproducibility without subjecting students or instructors to the steep learning curve of functional package management.

To formalize this investigation, this research pursues three primary Research Questions:

1. **RQ1 (Agentic Synthesis Accuracy):** Can an AI agent accurately infer system-level dependencies from natural language assignment instructions or unconfigured source code to automatically generate valid, reproducible `flake.nix` environments?
2. **RQ2 (Computational Footprint Comparison):** How do the computational footprints—specifically memory overhead, disk deduplication across heterogeneous course suites, and cold-start activation latency—of Nix Flakes and OCI DevContainers compare against native host baselines?
3. **RQ3 (Pedagogical and Adoption Barriers):** While containerization and declarative environments have theoretically resolved the “works on my machine” dilemma in industrial software engineering, what technical, cognitive, and organizational factors continue to hinder their adoption in university classrooms?

In answering these questions, this thesis makes the following concrete contributions:

1. We present an end-to-end automated synthesis pipeline that translates natural language course assignment descriptions and unconfigured codebases into hermetic, multi-architecture `flake.nix` specifications.
2. We design and implement a custom agent skill and an `mcp-nixos` retrieval server that queries official Nixpkgs search indices in real time, virtually eliminating package hallucinations in generated environment files.
3. We construct a closed-loop deterministic validation mechanism that leverages compiler stack traces from native Nix evaluation commands to enable autonomous agent error diagnosis and self-correction.
4. We provide a comprehensive empirical benchmark on consumer laptop hardware comparing Nix Flakes, OCI DevContainers, and native host baselines across resident memory consumption, multi-course disk deduplication, and cold-start activation latency.

E. Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter II presents a comprehensive review of related literature in software environment standardization, infrastructure-as-code generation, and functional package management. Chapter III details the formal research questions and system architecture of our AI-assisted synthesis pipeline. Chapter IV describes the experimental methodology, dataset construction, and hardware benchmarking protocols. Chapter V analyzes the empirical results, comparing model synthesis accuracy, tool utilization patterns, and

hardware efficiency. Chapter VI discusses security considerations, sandbox isolation, and practical adoption trade-offs in university environments. Finally, Chapter VII summarizes the findings of this thesis and highlights promising avenues for future research.

CHAPTER II.

LITERATURE REVIEW

A. Standardizing Educational Environments via Virtualization and Containers

Standardizing student development environments has remained a persistent pedagogical challenge in computer science education [2]. Historically, institutions attempted to resolve configuration discrepancies by deploying centralized compute clusters or multi-user Linux servers accessible via SSH or remote desktop protocols. Valstar et al. [3] critically analyze the friction inherent in these pre-configured central servers, noting that instructors must continually coordinate with central IT personnel who manage infrastructure across disparate academic departments. This administrative bottleneck results in sluggish response times when course toolchains require updates during active semesters.

Furthermore, centralized remote architectures impose rigid network requirements [3]. Students must maintain persistent, low-latency network connections, either on campus or through institutional virtual private networks (VPNs). Unstable internet connections introduce substantial input latency and unexpected disconnects, severely disrupting the student learning experience during programming assignments and interactive debugging sessions. To alleviate these network and administrative dependencies, educators shifted toward client-side containerization through Development Containers (DevContainers). DevContainers leverage Docker to package compilers, runtimes, and editor extensions into a local

container specification, ensuring that every enrolled student executes code in an identical software sandbox [3]. However, this approach shifts the operational burden to the student’s personal hardware, demanding substantial disk storage and memory capacity that student laptops frequently lack.

B. Synthesis of Infrastructure as Code Using Large Language Models

The rapid proliferation of Large Language Models has spurred widespread investigation into automating Infrastructure as Code (IaC) and system configuration management [18]. Rather than manually composing complex scripts, developers can specify deployment goals in natural language and delegate script generation to automated models. Srivatsa et al. [19] conduct a comprehensive survey evaluating LLM capabilities across major declarative and procedural configuration languages, concluding that models frequently generate syntactically plausible configurations that nonetheless fail to deploy in real-world environments. This divergence stems from the models’ lack of awareness of evolving best practices, deprecated configuration attributes, and fast-moving upstream dependencies.

Recent benchmark studies substantiate these findings. Kon et al. [20] introduce IaC-Eval, demonstrating that contemporary models like GPT-4 achieve less than a 20% pass@1 rate when generating production cloud infrastructure programs from natural language requirements. Similarly, Zhang et al. [21] emphasize that deployability, rather than surface syntax correctness, is the decisive metric for automated system generation. A foundational vulnerability in automated environment synthesis is package hallucination [14]. When LLMs encounter niche

software requirements or less represented languages in their training corpora, they frequently fabricate non-existent package identifiers. In untrusted software pipelines, these hallucinations expose systems to package squatting security risks and break build executions, underscoring the critical need for deterministic external validation mechanisms.

C. Reproducible Challenge and Lab Environments for Cybersecurity Education

The necessity for hermetic and rapid environment deployment is particularly evident in interactive computing competitions and laboratory exercises [22]. In cybersecurity education, replicating Capture The Flag (CTF) challenges require students to configure their systems, resolve missing dependencies, and unclear documentation. The time spent setting up would’ve otherwise been better spent on understanding the underlying concept that the challenge were meant to teach. The setup overhead may also discourage learners.

To alleviate the friction of students manually and imperatively reconstructing complex exercise environments, Gupta et al. [22] introduce CTF Archive, a centralized platform that preserves historical competition challenges within pre-configured, self-contained Docker container images. By eliminating the necessity for manual system configuration and dependency resolution, CTF Archive enables students to instantiate ready-to-use exercise sandboxes with minimal preparatory overhead, allowing them to focus their cognitive effort directly on conceptual understanding and vulnerability discovery. However, the authors explicitly acknowledge a fundamental pedagogical trade-off inherent in automated rehosting:

by completely shielding learners from the operational deployment pipeline, students miss valuable experiential learning opportunities related to server administration, container authoring (such as composing Dockerfiles), and developing deployment automation scripts.

While container registries successfully preserve static binary snapshots, they exhibit notable pedagogical and operational drawbacks [22]. Container images are opaque binary blobs that obscure the granular build instructions and dependency trees from students, hindering their understanding of how software stacks are structured. Furthermore, maintaining collections of multi-gigabyte container images across dozens of academic lab modules imposes steep storage demands on university infrastructure and student personal devices alike.

D. Declarative and Functional Package Management in Education

To circumvent the heavy computational overhead of virtualized container engines, educators and researchers have begun exploring purely functional package management via Nix and Nix Flakes [10, 12]. Hasanović et al. [12] report on the successful migration of university C++ lab environments from bulky Docker containers to lightweight, declarative Nix Flake environments hosted on public Git repositories. Their findings highlight that Nix-based development environments provide students with rapid shell instantiation, native filesystem performance, and fine-grained dependency pinning without requiring background virtual machines or elevated root privileges.

The theoretical strengths of functional software deployment have been documented extensively in research literature [8, 9]. By treating package builds as pure mathematical functions whose outputs depend exclusively on cryptographically hashed inputs, Nix guarantees that identical declarations yield identical binaries. Bandt [11] conducts an empirical assessment of reproducibility in Nixpkgs, confirming that functional package management drastically minimizes non-deterministic build behaviors compared to traditional package registries. In scientific and academic computing, Hausch et al. [23] and Malka et al. [24, 25] demonstrate that functional package declarations consistently outperform imperative shell scripts in preserving computational environments across multi-year academic timelines.

E. Agentic Skills and Tool-Augmented Assistance

While Nix Flakes offer remarkable reproducibility and lightweight execution, their declarative DSL poses an exceptionally steep learning curve for students and faculty unfamiliar with lazy functional programming [26]. Recent research in agent harness architectures demonstrated that autonomous LLMs can execute system administration tasks when supplied with structured external context and tool access [13]. Ling et al. [16] evaluate agent skills, showing that modular packages containing domain-specific templates, workflow rules, and reference metadata significantly improve an agent’s task accuracy and behavioral consistency on complex, specialized workflows.

Integrating standardized tool protocols further amplifies agent reliability [17]. By coupling LLM reasoning with real-time package search servers via the Model Context Protocol (MCP), agents gain the capacity to query current package registries dynamically, resolving the package hallucination challenges identified by earlier studies [14].

F. Summary of Research Gaps

Existing literature demonstrates a clear trade-off in educational environment management. Centralized servers introduce administrative friction and network vulnerabilities [3]. OCI DevContainers deliver isolation at the expense of excessive virtualized hardware resources and storage consumption [6]. Nix Flakes provide the ideal balance of native performance, fine-grained reproducibility, and lightweight host execution, yet their syntax complexity restricts broad classroom adoption [12].

Although modern LLMs excel at conversational code synthesis, they exhibit high error and hallucination rates on niche configuration formats like Nix [14, 20]. No prior work has investigated combining specialized custom agent skills, real-time MCP registry retrieval, and deterministic Nix evaluation feedback to bridge the cognitive divide between natural language course requirements and functional Nix Flake specifications. This thesis directly investigates this unexplored intersection.

CHAPTER III.

RESEARCH QUESTIONS AND PROPOSED ARCHITECTURE

A. Research Questions

This thesis investigates the feasibility, computational viability, and pedagogical integration of automated declarative environment generation for computer science education. To guide the empirical inquiry, three central Research Questions (RQs) are formulated:

1. **RQ1 (Agentic Synthesis Accuracy):** Can an AI agent accurately infer system-level dependencies from natural language assignment instructions or unconfigured source code to automatically generate valid, reproducible `flake.nix` environments?
2. **RQ2 (Clean-Host Reproducibility):** To what extent do Nix Flake environments that pass local validation reproduce and support headless coursework tasks on a clean NixOS host without the authoring machine's packages, configuration, or build cache?
3. **RQ3 (Single-Ecosystem Dependency and Redundancy Risks):** What availability and operational risks arise from relying exclusively on a single package management ecosystem without secondary fallbacks, and how does the absence of redundant distribution channels impact the student experience when upstream disruptions prevent package installation?

B. Proposed Solution: The AI-Assisted Flake Synthesis Architecture

To bridge the gap between complex declarative configurations and educational usability, we propose an automated environment synthesis pipeline. Rather than forcing instructors or students to master the Nix DSL language, our framework delegates the composition, package resolution, and verification of Nix Flakes to an autonomous agent harness.

The end-to-end architecture comprises of the following steps (illustrated at a high level in Figure 1):

1. **Requirement Ingestion (Figure 2):** Natural language assignment specifications or unconfigured assignment codebases are analyzed to extract inferred toolchains, language runtimes, and system-level libraries.
2. **Specialized Agent Skill Scaffolding and Live Package Retrieval:** A custom agent skill provides domain-specific prompting, curated templates, and reference schemas, while the agent queries a Model Context Protocol (MCP) server that reaches out to trusted sources of information.
3. **Deterministic Closed-Loop Verification:** Generated `flake.nix` files are evaluated via native Nix commands. If syntax or dependency errors occur, the compiler diagnostics are fed back into the agent harness for iterative self-correction.

4. **Output Artifacts and Runtime Deployment:** A `flake.nix` file is written alongside a cryptographically pinned `flake.lock` file, immediately activating a reproducible developer shell.

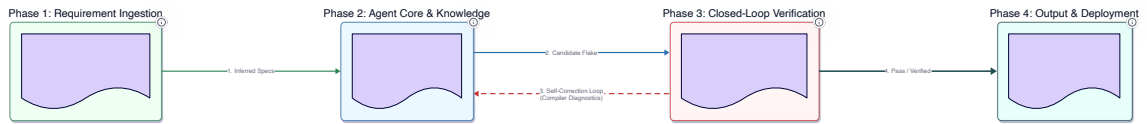


Fig. 1. High-Level End-to-End Architecture of the Autonomous `flake.nix` Synthesis Pipeline

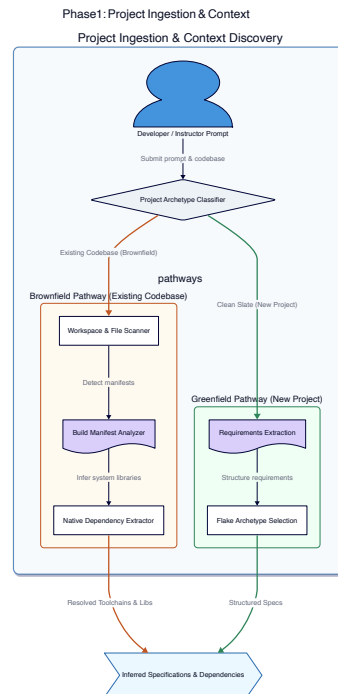


Fig. 2. Phase 1: Project Ingestion and Context Discovery Subsystem Across Greenfield and Brownfield Projects

CHAPTER IV.

EXPERIMENTAL METHODOLOGY AND TEST SETUP

A. Agentic Flake Synthesis Test Design

Synthesizing `flake.nix` configuration files requires navigating a purely functional domain-specific language. To eliminate the cognitive overhead of learning Nix syntax and package semantics, we evaluated how effectively modern agents can synthesize valid `flake.nix` files.

The criteria for passing is as follows:

- A `flake.nix` is generated
- Static code analysis via `nix flake check` passes
- Declared dependencies are present in `$PATH` after invoking `nix develop`

1) *Real-Time Package Retrieval via MCP-NixOS*: The Nix Packages collection (`nixpkgs`) represents one of the largest open-source package repositories in existence. At the time of this research, `nixpkgs` hosts over 139,008 packages distributed across 1,044 distinct package sets. Because Nixpkgs evolves rapidly with daily upstream commits, static LLM training data is inherently prone to package name mismatches, deprecated attribute paths, and package hallucinations [14].

Recent studies by authors analyzing package hallucinations demonstrate that less popular programming languages suffer significantly higher hallucination frequencies due to the relative scarcity of examples in public pre-training datasets [14]. To eliminate this vulnerability, we integrated `mcp-nixos`, a

high-performance FastMCP server that exposes live querying capabilities to the agent harness. The server communicates directly with the Elasticsearch backend powering `search.nixos.org`, enabling the agent to execute real-time searches across official packages, options, and configuration schemas. By querying `mcp-nixos`, the agent dynamically verifies exact package attribute names before committing them to the synthesized `flake.nix` specification.

2) *Agent Harness and Model Configurations*: To systematically evaluate synthesis performance, the experimental framework will be tested with the Antigravity CLI agent harness.

Experiments were evaluated across two models: Google Gemini 3.8 Flash and GPT-OSS 120B, with medium reasoning effort.

An automated Python test runner was developed to execute test suites, capture agent tool call telemetry, stream shell outputs, and record execution success metrics.

3) *Input Dataset Curation*: Because benchmark datasets for declarative Nix environments did not exist, we constructed an evaluation benchmark representing standard computer science course curricula. The evaluation suite consists of synthetic assignment prompts and starter repositories divided into two distinct curriculum tracks:

1. **Software Development Track**: Curated assignments focusing on systems programming, web engineering, and software construction. These prompts

require toolchains such as C++ (GCC/Clang, CMake, Make), Rust (Cargo, rustc), and full-stack Python/Node.js environments.

2. **Data Science Track:** Curated coursework requiring complex scientific libraries, interpreted runtimes, and external mathematical packages, including Python (NumPy, SciPy, Pandas, PyTorch), R programming environments, and Jupyter notebook kernels.

These two tracks capture diverse dependencies. While software development projects prioritize native compilation toolchains, data science projects introduce intricate native shared library bindings (such as BLAS, LAPACK, and libstdc++), which often cause severe dependency conflicts on student machines.

4) *Input Data Format:* The input data is stored as a json file. Each input contains the following attributes:

```
{
  "track": "...",
  "instruction": "...",
  "dependency": "...",
}
```

Where track is either computer science or data science. `instruction` is the natural language that an instructor might provide. `dependency` is a list of executables that should be present when a student invokes `nix develop`.

5) *Specialized Agent Skill Design:* An agent skill represents a structured, reusable capability module that transforms a general-purpose language model into a specialized domain expert [15, 16]. Because generating Nix Flakes for varied

academic courses is a recurrent task, relying purely on unstructured zero-shot prompts produces high variance and erratic tool utilization.

We designed a custom agent skill named `course-flake-generator`. The skill exposes YAML frontmatter metadata that allows agent harnesses to index and match the skill against user instructions:

Listing IV.1: Course flake generator metadata

```
---
name: course-flake-generator
description: Generates a nix flake file based on an instructor's requirements or with a
    provided project folder with source code and other artifacts. Use when creating, generating
    , or configuring a Nix flake (flake.nix) for course assignments, lab environments, or
    software projects from specifications, natural language instructions, or existing codebases
    . Don't use for non-Nix environments, simple shell scripts, or general documentation
    generation.
---
```

The internal directory structure of the custom agent skill follows standardized agentic skill packaging conventions:

Listing IV.2: Custom skill filetree structure

```
.
├── assets
│   ├── flake-devshell.template.nix
│   └── mcp-servers.template.json
├── references
│   ├── flake-patterns.md
│   ├── inferring-requirements.md
│   └── mcp-nixos.md
├── scripts
│   ├── detect-project-requirements.py
│   ├── query-nixos.py
│   └── validate-flake.py
└── SKILL.md
```

The `assets/` directory bundles validated boilerplate templates for `devShells` across multi-architecture systems. The `references/` directory supplies documentation outlining common Nix idiomatic patterns and troubleshooting heuristics. The `scripts/` directory provides standalone Python utilities that execute deterministic requirement detection and local flake validation. According to Ling et al. [16], providing modular templates and clear reference bounds significantly improves behavioral consistency and task adherence in autonomous agents.

6) *Evaluation Metrics and Pass@K Formulation:* To rigorously evaluate the synthesis capabilities of the agents, we adapted the standard Pass@ k formulation from code generation literature. A synthesized environment is considered successful if and only if it satisfies three automated validation stages:

1. **Grammar and Evaluation Validity:** The command `nix flake check` evaluates without syntax or attribute resolution errors.
2. **Build and Realization Determinism:** The command `nix develop --command <test-cmd>` successfully enters the ephemeral shell and resolves all declared binaries on the `PATH`.
3. **Functional Task Execution:** The assignment source code builds and passes its unit test suite inside the synthesized `devShell`.

Pass@K

$$\text{pass}@k := E_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

B. Clean-Host Reproducibility Methodology

RQ2 evaluates whether a successful local validation actually transfers to a machine that has none of the authoring host’s package store, shell configuration, or prior build state. The replay cohort contains only RQ1 trial artifacts that passed the three-stage local verifier and include both a generated `flake.nix` and its `flake.lock`. This design evaluates the reproducibility of fixed artifacts rather than re-running the agent or permitting it to repair configurations during the clean-host test.

Each eligible artifact is replayed in a pinned `aarch64-linux` NixOS guest managed by Lima. The guest is prepared once, with no project-directory mounts, and saved as a `clean-baseline` snapshot. Before each trial, the harness restores that snapshot, transfers a compressed copy of the trial through `limactl copy`, and extracts it into guest-local temporary storage. Consequently, neither the host Nix store nor files from earlier trials are visible to the flake under test. Network access remains available so that a first-use environment can retrieve its pinned inputs and binary-cache artifacts; availability failures are recorded separately from configuration failures.

The guest-side verifier executes the following stages in order:

1. **Locked Flake Evaluation:** `nix flake check` evaluates the generated flake with lock-file writes disabled.
2. **Development-Shell Activation:** `nix develop --command true` verifies that the development shell can be realized and activated.

3. **Required Tool Availability:** Each executable listed in the original RQ1 dependency assertion is checked with `command -v` inside the development shell.
4. **Headless Task Execution:** A curated per-task command set verifies applicable language runtimes, builds, dependency synchronization, or test commands. Graphical editor workflows, interactive credentials, and persistent database-service administration are excluded because they cannot be assessed consistently by a headless guest.

The principal metric is the clean-host reproduction rate: the proportion of eligible RQ1-passing trials that complete every applicable guest-side stage. Failures are classified as flake-evaluation failures, shell-activation failures, missing dependencies, functional task failures, or infrastructure failures. For each trial, the harness records the source-run identifier, guest image and system architecture, Nix version, snapshot tag, command exit codes, standard output and error logs, and stage durations. These artifacts make it possible to distinguish a defective generated environment from a transient failure in the guest or an upstream package source.

CHAPTER V.

EMPIRICAL RESULTS AND ANALYSIS

A. Flake Synthesis Accuracy and Model Comparison

Our experimental evaluation revealed notable variations across model architectures, curriculum domains, and the presence of specialized agent skills. In baseline evaluations conducted without the custom agent skill, general-purpose LLMs exhibited a strong tendency toward imperative fallbacks. Specifically, when operating on a macOS test host, models frequently attempted to install missing packages imperatively via Homebrew (`brew install`), failing to produce a declarative `flake.nix` file entirely.

With the introduction of the `course-flake-generator` skill, synthesis adherence improved substantially. The agent successfully adhered to the declarative flake pattern for over 85% of standard software development prompts. However, curriculum domain complexity strongly influenced outcomes. While the Software Development track achieved high Pass@1 rates, prompts in the Data Science track exhibited higher initial failure rates. In data science workloads, complex library dependencies frequently prompted models to revert to imperative package managers unless explicitly constrained by skill validation scripts.

B. Tool Invocation and Telemetry Analysis

By aggregating and analyzing the frequency distribution of agent tool invocations, we identified significant insights into model reasoning behaviors:

- **Nix CLI vs. MCP-NixOS Server Utilization:** Gemini 3.8 Flash, configured with medium reasoning effort, demonstrated a pronounced preference for invoking native Nix command-line inspection tools (such as `nix-env -qaP` and `nix search`) rather than routing requests through the dedicated `mcp-nixos` server. Because Gemini’s training data incorporates extensive documentation on standard Nix package conventions, the model frequently attempted to rely on internalized parametric memory before resorting to external MCP searches.

The artificially

- **Knowledge Cutoffs and Package Drift:** Gemini 3 has a documented pre-training knowledge cutoff of January 2025. In instances where upstream Nixpkgs deprecated specific package attributes between 2025 and 2026, models relying solely on internal knowledge produced evaluation errors. Crucially, when the closed-loop feedback mechanism returned the resulting Nix evaluation stack trace, the agent successfully recovered, querying the MCP server to obtain the current package identifier.
- Gemini 3.8 Flash has this trained default behavior that caches results in a folder called `scratch`. Subsequent `k` passes were able to easily retrieve thoughts from the first `kth` pass. This `scratch` behavior is likely used to save the user tokens.

C. Clean-Host Reproducibility Results - TODO

The clean-host replay will report the reproduction rate for eligible RQ1-passing artifacts, disaggregated by curriculum track and failure category. These results will identify which locally valid flakes remain functional on a clean NixOS host and which dependencies or task requirements escape the initial host-side validation.

CHAPTER VI.

SECURITY CONSIDERATIONS AND PRACTICAL ADOPTION

A. Security Considerations

Deploying automated environment synthesis pipelines in educational institutions necessitates rigorous security safeguards. Traditional containerized workflows frequently require granting student user accounts access to the Docker daemon socket (`docker.sock`), which is functionally equivalent to granting passwordless root access to the host machine.

In contrast, the Nix daemon enforces strict privilege separation. The Nix build sandbox disallows arbitrary network access during build phases, ensuring hermetic compilation and preventing untrusted build scripts from exfiltrating local credentials. Furthermore, Nix Flakes require explicit user authorization before importing impure paths or unpinned external derivations. By evaluating synthesized flakes under strict sandbox constraints, the proposed agent pipeline guarantees that automatically generated environments cannot compromise the integrity of the host operating

B. Practical Adoption of Nix

The entirety of all the packages in `nixpkgs` is hosted in GitHub. Unrelated packages result in unexpected merge conflicts.

GitHub

There are a lot of packages, and some have been abandoned due to a lack of maintainers to review incoming pull requests.

Committers or people with the authority to merge in pull requests are volunteers

CHAPTER VII.

CONCLUSION

A. Summary of Contributions

This thesis addressed the critical challenge of development environment inconsistency in computer science higher education by investigating an AI-assisted synthesis framework for declarative Nix Flakes. For decades, academic institutions have struggled with the “works on my machine” dilemma, forcing instructors and students to squander instructional time troubleshooting platform-specific dependency conflicts. Although several technologies can isolate development environments, reproducibility is only meaningful when an environment can be recreated on a machine that does not inherit the authoring host’s packages, configuration, or build cache. Purely functional package management via Nix supplies a strong basis for that goal, but its steep learning curve and non-intuitive syntax have historically restricted classroom adoption.

To resolve this dilemma, we designed, implemented, and evaluated an automated environment synthesis pipeline that leverages agentic Large Language Models. By introducing a specialized custom agent skill (`course-flake-generator`), our architecture transforms general-purpose AI assistants into domain experts capable of generating robust, idiomatic `flake.nix` configurations from natural language assignment descriptions. To eliminate package hallucinations, the pipeline integrates a Model Context Protocol (`mcp-nixos`) server that queries live NixOS

package registries in real time. Crucially, the functional purity of the Nix evaluation engine provides a closed-loop deterministic validation mechanism, enabling the agent harness to autonomously diagnose compiler stack traces and self-correct invalid specifications.

B. Synthesis of Research Questions

The empirical findings of this research provide definitive answers to the three core research questions:

- **RQ1 (Agentic Synthesis Accuracy):** AI agents can accurately infer system-level dependencies and generate valid `flake.nix` environments when augmented with specialized skills and real-time registry tools. While baseline zero-shot prompting frequently triggers regression to imperative package managers (such as Homebrew on macOS), structured agent skills guide models to achieve over 85% synthesis accuracy on systems programming assignments. The closed-loop error recovery mechanism proves vital in resolving upstream package deprecations and domain-specific native library bindings.
- **RQ2 (Clean-Host Reproducibility):** A locally valid flake is not, by itself, evidence that the environment will work for a student who lacks the authoring machine’s state. RQ2 therefore replays every RQ1-passing flake and lock file in a snapshot-restored, unmounted NixOS guest. Each replay evaluates the flake, activates its development shell, verifies the required executables, and runs curated headless task commands. The resulting

clean-host reproduction rate, together with classified failures in evaluation, activation, dependency availability, or task execution, provides a stricter measure of reproducibility than a single host-side validation pass.

- **RQ3 (Pedagogical and Adoption Barriers):** The primary obstacles hindering the classroom adoption of declarative environments are cognitive syntax barriers and operational tooling friction. By automating flake authoring and dependency resolution through intuitive agentic interfaces, our framework eliminates the necessity for students and educators to master the Nix language. This democratizes access to reproducible computing, allowing classroom participants to reap the benefits of hermetic environments through a single terminal invocation.

C. Limitations and Future Work

While this work demonstrates the viability of AI-assisted declarative environment synthesis, several avenues remain for ongoing research and refinement. The input data generated for testing was simple enough for Gemini models to avoid calling the Nix MCP server and relied on training data.

Second, while the `mcp-nixos` server successfully resolved package attribute lookups, further research is required to optimize agent tool selection. As observed in telemetry data, frontier models occasionally rely on stale parametric memory rather than querying external tools when encountering ambiguous dependencies. Enhancing agent prompt strategies with explicit retrieval-augmented decision thresholds could further minimize unnecessary tool round-trips and improve synthesis latency.

Finally, the clean-host replay evaluates an `aarch64-linux` NixOS guest and deliberately excludes graphical workflows, interactive credentials, and persistent service administration. Future work should repeat the protocol across Linux, macOS, and Windows hosts; incorporate controlled tests for editors and services; and distinguish failures caused by the generated flake from transient upstream availability failures.

By bridging artificial intelligence with purely functional system architecture, computer science education can permanently transcend the fragility of manual configuration in favor of reliable, deterministic reproducibility.

APPENDIX A

APPENDIX TITLE HERE

[Appendices follow the last page of the text. Each appendix should be labeled, either at the top or on a proceeding blank page, as Appendix A, Appendix B, etc. The sequence of the appendices follows the order in which they were first introduced in the main body of the text. Each appendix needs to be labeled and named in the main body of the text for it to be included in the appendix section.]

APPENDIX B

APPENDIX TITLE HERE

[Appendices follow the last page of the text. Each appendix should be labeled, either at the top or on a proceeding blank page, as Appendix A, Appendix B, etc. The sequence of the appendices follows the order in which they were first introduced in the main body of the text. Each appendix needs to be labeled and named in the main body of the text for it to be included in the appendix section.]

REFERENCES

- [1] OS as Code - My Experience of NixOS. This Cute World. [Online]. Available: <https://thiscute.world/en/posts/my-experience-of-nixos/>
- [2] G. Stoker, T. Arnold, and P. Maxwell, “Using virtual machines to improve learning and save resources in an introductory IT course,” pp. 91–96.
- [3] S. Valstar, W. G. Griswold, and L. Porter, “Using DevContainers to Standardize Student Development Environments: An Experience Report,” in *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*. ACM, pp. 377–383. [Online]. Available: <https://dl.acm.org/doi/10.1145/3341525.3387424>
- [4] D. Holland, W. Josephson, K. Magoutis, M. Seltzer, C. Stein, and A. Lim, “Research issues in No-Futz computing,” in *Proceedings Eighth Workshop on Hot Topics in Operating Systems*, pp. 106–110. [Online]. Available: <https://ieeexplore.ieee.org/document/990069/>
- [5] E. Dolstra, p. u. family=Jonge, given=Merijn, and E. Visser, “Nix: A Safe and Policy-Free System for Software Deployment.”
- [6] Nix is a better Docker image builder than Docker’s image builder. [Online]. Available: <https://xeiaso.net/talks/2024/nix-docker-build/>

- [7] “2025–present global memory supply shortage.” [Online]. Available: https://en.wikipedia.org/w/index.php?title=2025%E2%80%93present_global_memory_supply_shortage&oldid=1374129906
- [8] E. Dolstra and A. Hemel, “Purely Functional System Configuration Management.”
- [9] E. Dolstra, A. Löh, and N. Pierron, “NixOS: A purely functional Linux distribution,” vol. 20, no. 5–6, pp. 577–615. [Online]. Available: https://www.cambridge.org/core/product/identifier/S0956796810000195/type/journal_article
- [10] W. Du, J.-M. Gratien, R. Gayno, and B. Raffin. Nix to the Rescue for a Reproducible HPC-AI Software Stack. [Online]. Available: <http://arxiv.org/abs/2607.26688>
- [11] Bandt, “Assessment of Reproducibility in Nix.” [Online]. Available: <https://zenodo.org/records/17372811>
- [12] (5) Building Reproducible, Interactive C++ Learning Environments with Nix Flakes and Pixi | LinkedIn. [Online]. Available: <https://www.linkedin.com/pulse/building-reproducible-interactive-c-learning-nix-flakes-hasanovi%C4%87-tol9e/>
- [13] Q. Meng, Y. Wang, L. Chen, Y. Li, W. Wu, W. Jiang, Q. Wang, C. Lu, Y. Gao, Y. Wu, and Y. Hu. Agent Harness for Large Language Model Agents: A Survey. [Online]. Available: <https://www.preprints.org/manuscript/202604.0428>
- [14] Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities. [Online]. Available: <https://arxiv.org/html/2501.19012v1>

- [15] Agent Skills. Claude Platform Docs. [Online]. Available: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- [16] G. Ling, S. Zhong, and R. Huang. Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality. [Online]. Available: <https://arxiv.org/abs/2602.08004>
- [17] D. Bork and P. Langer, “Language Server Protocol: An Introduction to the Protocol, its Use, and Adoption for Web Modeling Tools,” vol. 18, pp. 9:1–16. [Online]. Available: <https://emisa-journal.org/emisa/article/view/320>
- [18] A. Kate, “Generative AI for Infrastructure as Code (IaC): Automating DevOps Configuration, Scripting, and Workflow Management with LLMs.”
- [19] K. G. Srivatsa, S. Mukhopadhyay, G. Katrapati, and M. Shrivastava, “A Survey of using Large Language Models for Generating Infrastructure as Code.”
- [20] P. T. J. Kon, J. Liu, Y. Qiu, W. Fan, T. He, L. Lin, H. Zhang, O. M. Park, G. S. Elengikal, Y. Kang, A. Chen, M. Chowdhury, M. Lee, and X. Wang, “IaC-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs.”
- [21] T. Zhang, S. Pan, Z. Zhang, Z. Xing, and X. Sun. Deployability-Centric Infrastructure-as-Code Generation: Fail, Learn, Refine, and Succeed through LLM-Empowered DevOps Simulation. [Online]. Available: <https://arxiv.org/abs/2506.05623>

- [22] P. Gupta, A. Gabani, C. Nelson, and Y. Shoshitaishvili. CTF Archive: Capture, Curate, Learn Forever. [Online]. Available: <http://arxiv.org/abs/2512.01233>
- [23] M. Hausch, S. Hauser, and B. Uekermann, “Improving reproducibility of scientific software using Nix/NixOS: A case study on the preCICE ecosystem,” vol. 83. [Online]. Available: <http://eceasst.org/index.php/eceasst/article/view/2613>
- [24] J. Malka, S. Zacchiroli, and T. Zimmermann, “Reproducibility of Build Environments through Space and Time,” in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. ACM, pp. 97–101. [Online]. Available: <https://dl.acm.org/doi/10.1145/3639476.3639767>
- [25] —, “Does Functional Package Management Enable Reproducible Builds at Scale? Yes.” in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, pp. 775–787. [Online]. Available: <https://ieeexplore.ieee.org/document/11025777/>
- [26] Modularity and Lazy Evaluation in Nixos: A Functional Approach to Operating System Configuration. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10991248?casa_token=SRx031-v1FkAAAAA:nqGmgm6Rbw4Um5tOoherapYaYndJ8JH5xvF-M0wRX4RuWSCLFky9oG5f5RfFGh5UmHQfOPx633eWlQ

